

Unit

2

Lesson

1



Innovators in Action.

Power of the Sense!

Time Frame: [80-90 min].

Ready, Set, Learn!

By the end of this lesson students will be able to:

- Explain how the brain sends electrical signals through nerves to control hand and finger movements.
- Describe how a flex sensor works and how its resistance changes when it is bent.
- Use a table or graph to represent how the bend angle affects the sensor's resistance.
- Design and draw a basic circuit that includes a flex sensor to detect finger movement.

Challenge questions of the lesson

- How does the brain use electrical signals to control movements in your fingers and hand?
- What happens inside a flex sensor when it bends, and how does this affect resistance?
- How can you use tables or graphs to show the relationship between bend angle and sensor resistance?
- What are the important steps in designing a circuit that detects finger movement using flex sensors?
- How might understanding sensors and circuits help you invent new assistive technologies?

SEL Relation

- **Self-Awareness:** Encouraging students to connect bodily sensations and movements to scientific concepts.
- **Growth Mindset:** Fostering persistence and problem-solving as students design circuits and debug code.
- **Collaboration:** Developing communication and teamwork skills during group building and coding activities.
- **Cultivating Empathy:** Increasing understanding of assistive technologies that help people with disabilities communicate and interact.

Engage



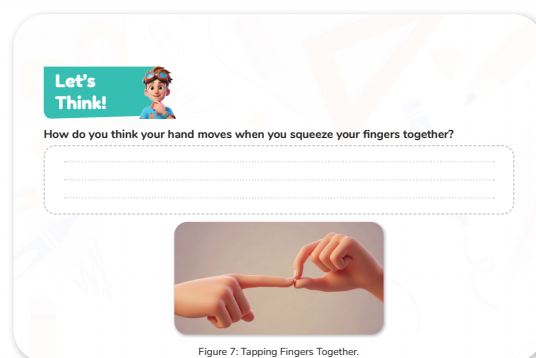
Teacher's Role:

- Read the story aloud with enthusiasm and clarity, emphasizing key concepts like conductivity and touch sensors.
- Use gestures and props (like copper tape or wires) to visually demonstrate what is being described.
- Pause to check for understanding and encourage questions from students.
- Facilitate the “Let’s Think” activity by asking the question aloud and prompting students to consider how their hands move when tapping fingers.
- Provide support or rephrase questions for students who may need it.
- Encourage all students to participate and share their ideas.

Expected from students:

- Listen attentively to the story and visualize the setup and experiments.
- Ask relevant questions to clarify understanding.
- Reflect on the “Let’s Think” question and share their observations about finger movement.
- Respectfully listen to classmates’ ideas and contribute their own thoughts clearly.

Parts of the book included.



Story



- Gather the class and create an atmosphere of curiosity and exploration.
- Say:
 - “Today, we’re going to explore a fascinating new kind of touch sensor—something that can feel when you tap your fingers together, just like a glove might. Let’s listen carefully to see how Adam, Laila, and Mrs. Sara discover this technology.”
- Read the story aloud with expression and pacing, emphasizing wonder and the discovery process.
- Pause at key points to ask guiding questions, such as:
 - “What is special about the tape Mrs. Sara shows?”
 - “How do you think the touch sensor works to send messages?”
 - “Why would it be useful for a glove to sense taps?”
- Encourage students to imagine themselves experimenting and discovering like the characters.
- Invite a few students to share quick predictions or questions before moving on.

Possible Strategies for Implementation:

1. Create an Engaging Introduction.

- Set a curious tone by highlighting the mystery and potential of touch sensors.
- Use relatable examples, like gloves or smartphones, to connect the concept to students’ lives.

2. Expressive and Dynamic Reading

- Vary your voice to differentiate characters and emphasize key ideas.
- Use deliberate pauses to let important information sink in and prompt reflection.

3. Use Visual Aids and Props

- Show actual copper tape or conductive materials if available, letting students touch and explore them.
- Use diagrams or images to illustrate how electricity flows when the tape is touched.

4. Ask Guiding Questions

- During reading, pause and ask questions such as:
 - “Why do you think conductivity is important for touch sensors?”
 - “How does touching two points create a message?”
- Encourage students to think critically and make predictions.

5. Relate to Prior Knowledge.

- Connect the story to previous lessons on communication and sensors.
- Invite students to share experiences with touchscreens or other sensor-based devices.

6. Encourage Visualization and Role Play.

- Ask students to visualize themselves connecting wires and testing sensors.
- Optionally, have volunteers act out parts of the story or demonstrate finger taps.

7. Clarify Vocabulary and Concepts.

- Explain terms like “conductive,” “signal,” and “sensor” using simple language and examples.
- Use analogies (e.g., electricity flowing like water) to support understanding.

8. Support Diverse Learners

- Provide sentence starters for responses.
- Use visuals and hands-on materials for students who benefit from multi-modal learning.

Let's Think!



Objective:

Students will develop awareness of the physical movements involved when tapping fingers together, enhancing observation skills and connecting body movement to touch sensor technology.

Sample Answers:

- “When I tap my fingers, my index finger moves down and touches my thumb, then they separate again.”
- “My hand feels a small quick movement at the tips of my fingers.”
- “It’s like a small click or touch that is very light but clear.”
- “The movement is controlled by the muscles in my fingers and wrist.”

Possible Strategies for Implementation:

1. Encourage Personal Reflection

- Ask students to close their eyes briefly and focus on how their fingers move during a tap.
- Invite them to describe the sensation or movement in their own words or through drawing.

2. Facilitate Pair or Small Group Discussion.

- Have students share their observations with a partner or small group to compare experiences.

- Encourage descriptive language and listening skills.

3. Use Guided Questions

- Prompt deeper thinking with questions like:
 - “What muscles or joints do you think are involved in the movement?”
 - “How precise do you think this movement needs to be for a sensor to detect it?”

4. Connect to Technology

- Link the physical movement discussion to how sensors detect touch or taps.
- Explain briefly how sensors respond to these specific finger movements.

5. Support Diverse Learners

- Allow alternative expressions such as drawing or demonstrating the movement.
- Provide sentence starters to scaffold verbal or written responses.

Explorer's Toolkit



Teacher's Role:

- Prepare devices or computer stations so all students can access and navigate the simulation smoothly.
- Introduce the activity by explaining that the simulation shows how the brain sends signals to move the hand.
- Provide clear instructions on how to use the simulation, including key controls and what to observe.
- Circulate during the activity to assist with technical issues and prompt deeper thinking by asking questions like, “What happens when you bend the hand this way?”
- Encourage students to explore different hand movements and notice how the brain controls each motion.
- Facilitate a follow-up discussion to share observations and connect the simulation to the real human body.

Expected from students:

- Follow instructions to navigate and interact with the simulation carefully.
- Observe how brain signals affect hand movement in the virtual model.
- Record observations or notes about how different movements are controlled.
- Participate actively in follow-up discussions, sharing insights or questions.
- Use technology responsibly and collaboratively.

Parts of the book included.

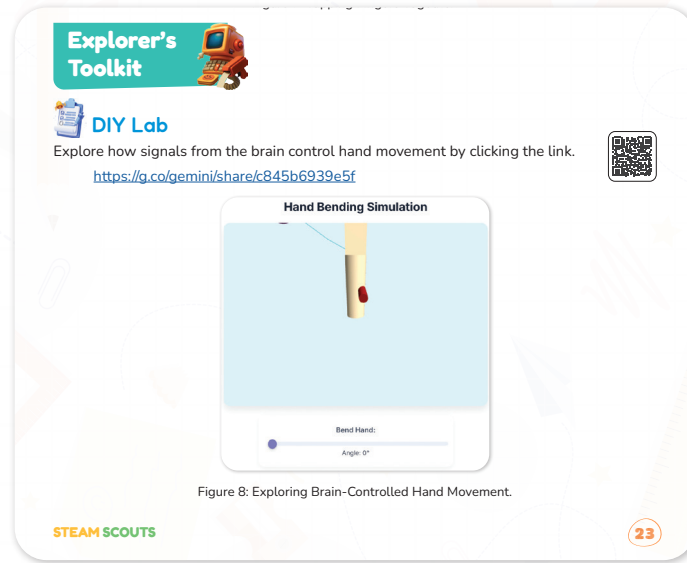


Figure 8: Exploring Brain-Controlled Hand Movement.

DIY Lab

Objective:

Students will investigate how signals from the brain control hand movement by interacting with a digital simulation, enhancing understanding of the nervous system and motor control.

Simulation Link: <https://g.co/gemini/share/c845b6939e5f>



Activity instructions:

1. Introduce the Concept

- Briefly explain how the brain sends electrical signals to muscles through the nervous system to create movement.

2. Launch the Simulation

- Display the link and explain what students will see and control (bending a digital hand using a slider to simulate signal strength).

3. Guide Observation

- Encourage students to manipulate the slider slowly and observe the hand's angle, response time, and flexibility.

4. Prompt Reflection

- Use the following guiding questions (on the board or printed):
 - What happens to the hand when the signal increases?
 - What might happen if the brain couldn't send signals?
 - How might this idea connect to creating assistive technology for someone who can't speak or move?

5. Connect to the Smart Glove Project.

- Emphasize how sensors in the glove might mimic the nervous system to detect and

translate movement into signals (e.g., text or sound).

- Tips:
 - Use this simulation as a transition into the design phase of the smart glove project.
 - Reinforce vocabulary: neurons, signals, flexion, motor control.
 - Allow fast finishers to sketch how they think a sensor might detect bending in a real glove.

Possible Strategies for Implementation:

1. Pre-Activity Preparation

- Briefly review the basics of the brain and nervous system before starting.
- Set a clear objective for the simulation activity.

2. Guided Exploration

- Demonstrate the simulation on a projector or shared screen.
- Highlight important features and controls.

3. Think-Aloud Modeling

- Model thinking aloud while interacting with the simulation to show how to observe and analyze movements.

4. Pair or Small Group Work.

- Allow students to work in pairs or small groups to encourage collaboration and shared learning.

5. Prompt Reflection

- Use guided questions during or after the activity, such as:
 - “How does the brain send signals to different parts of the hand?”
 - “What did you notice about how the fingers move?”
 - “Why is this process important for everyday tasks?”

6. Support and Differentiation

- Provide written or visual step-by-step guides for students needing extra support.
- Offer alternative ways to document learning (drawing, verbal summary).



Teacher's Role:

- Prepare the classroom technology to ensure clear video playback for all students.
- Introduce the video by explaining that it will show how a flex sensor detects bending and sends signals.

- Set a clear viewing purpose by asking students to focus on how the sensor changes as it bends.
- Pause the video at important points if needed to clarify concepts or answer questions.
- Facilitate a post-video discussion to check comprehension and relate the sensor's function to the lesson's hands-on activities.
- Support students who need help by rephrasing or providing examples.

Expected from students:

- Watch the video attentively, focusing on how a flex sensor operates.
- Take notes or mentally track key points about sensor function and applications.
- Ask questions to clarify understanding.
- Participate actively in post-video discussion, connecting concepts to real-world examples.

Parts of the book included.





Screen Lessons

Click on the link to learn more about what is flex sensor and how it works?!

https://youtu.be/BEpylwiw_xo?si=v1G90LJ-o54EbAAf



Screen Lessons

Objective:

Students will understand what a flex sensor is and how it works by watching a detailed video, building foundational knowledge of sensor technology relevant to wearable devices.

Video Link: https://youtu.be/BEpylwiw_xo?si=v1G90LJ-o54EbAAf



Possible Strategies for Implementation:

1. Pre-Viewing Preparation

- Activate prior knowledge by asking students if they have ever used or seen devices that respond to bending or movement (like game controllers, smartphones, or wearable fitness trackers).
- Explain basic sensor concepts in simple terms to prepare students for the video content.
- Pose a guiding question such as, "What do you think happens inside a sensor when it bends?"

2. Active Video Viewing

- Play the video in full once to give students an overall understanding.
- Replay key segments that demonstrate the flex sensor bending and how its electrical resistance changes.

- Pause to highlight important details and check for student understanding, encouraging questions or clarifications.

3. Use of Visual and Hands-On Supports.

- Supplement the video with diagrams or physical samples of flex sensors if available, so students can visualize and physically explore the concept.
- Demonstrate or show close-up images of how a flex sensor bends and connects to circuits.

4. Facilitated Discussion and Reflection.

- After viewing, ask students to explain how the sensor works in their own words.
- Encourage them to discuss why such sensors might be useful in wearable technology or communication devices.
- Connect this discussion to prior lessons on touch and signal transmission.

5. Differentiation for Diverse Learners.

- Provide vocabulary lists or flashcards with key terms like “flex sensor,” “resistance,” and “signal” with simple definitions and images.
- Offer sentence starters or graphic organizers to help students structure their understanding and explanations.
- Allow verbal, visual, or written responses according to student needs and preferences.

6. Encourage Inquiry and Curiosity.

- Challenge students to think about other ways sensors could be used in everyday life.
- Invite them to brainstorm possible new inventions or improvements using flex sensors.

Elaborate



Teacher's Role:

- Introduce the bending resistance formula clearly and explain how students will use it to calculate resistance values.
- Guide students in completing the table, offering support with calculations as needed.
- Demonstrate how to use the Canva Graph Maker or similar tool to plot their data.
- Provide examples of interpreting graphs and distinguishing between correct and incorrect data representations.
- Explain the purpose of building the circuit with flex sensors and Arduino, linking the action (hand movement) to reaction (displayed messages).
- Assist students in planning their coding logic by discussing what instructions the code should include.
- Encourage creative thinking about the types of messages that could be displayed.

- Monitor and support hands-on work, troubleshooting technical issues and promoting collaboration.
- Motivate students to apply their coding knowledge to complete the smart glove program.
- Provide scaffolded support with coding syntax and logic without giving direct answers.
- Encourage peer review and sharing of coding solutions.
- Lead a debrief session where students discuss their findings from the graphs and circuit building.
- Highlight connections between the scientific concepts of resistance, sensor input, and digital output.

Expected from students:

- **During Apprentice Assessment:**
 - Follow instructions carefully to calculate and record resistance values.
 - Use digital tools to graph data accurately.
 - Analyze graphs critically to identify which one best represents their data.
- **During Builder Activity:**
 - Participate actively in building and coding the flex sensor circuit.
 - Collaborate effectively with peers, sharing ideas and troubleshooting challenges.
 - Think creatively about the messages their circuit could display.
- **During Showcase Coding Task:**
 - Apply coding concepts to complete the program that makes the smart glove work.
 - Share their solutions and learn from others' approaches.
- **General:**
 - Engage respectfully in discussions and reflections.
 - Demonstrate curiosity and persistence when solving problems.

Parts of the book included.

Assessment

Apprentice

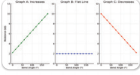
Let's discover how bending changes resistance!
Use the flex sensor formula below to complete the table:
Resistance (kΩ) = 2.2 + (Bend Angle + 180) × 7.8

Attempt	Estimated Bend (Degrees)	Expected Resistance (kΩ)
1	0	
2	30	
3	45	
4	90	
5	105	
6	130	
7	145	
8	180	

Time to Graph It!
Use the Canvas Graph Maker to create a line graph:
Label the X-axis as Bend Angle (°).
Label the Y-axis as Resistance (kΩ).
Start here: <https://www.canvasmaker.com/graph/>

You've seen how resistance changes as the flex sensor bends.
Now look at the three graphs below—only one matches your actual results!

1. Circle the graph that best shows your data.
2. Explain why the other two graphs don't seem correct. What's off about them?



Builder

Let's explore how your hand movements (the action) can create a visible message (the reaction)!

In this activity, you'll design and build a basic circuit that connects multiple flex sensors to an Arduino. Each time you bend a finger, the flex sensor will detect the motion and send a signal to the Arduino. You'll see how movement gets turned into data—just like magic!

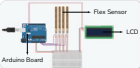


Figure 3: Arduino Setup with Ultrasonic Sensors and LCD Display

Pyramakerz

When you write the code, what kinds of instructions would you give it?

What kinds of messages could be shown if this circuit was connected to a screen?



Apprentice

Objective:

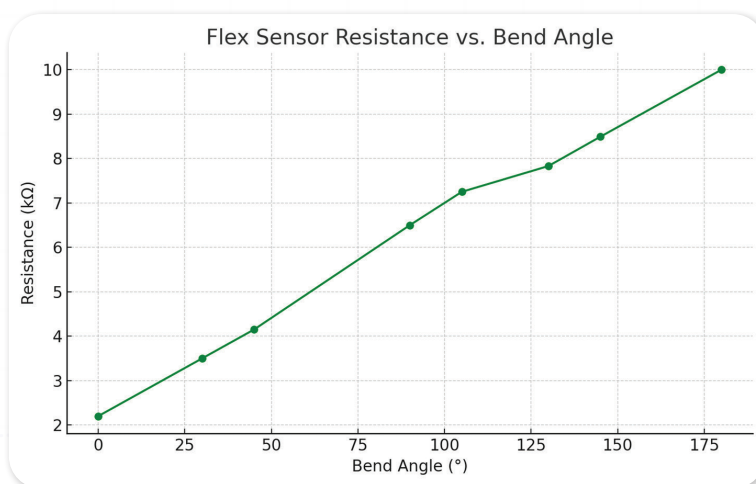
Students will apply a mathematical formula to calculate the resistance of a flex sensor at various bend angles, create a graph to visualize the relationship, and analyze which graph best represents their data by using reasoning skills.

Answer:

Using the formula:

$$\text{Resistance (k}\Omega\text{)} = 2.2 + (\text{Bend Angle} \div 180) \times 7.8$$

Attempt	Estimated Bend (Degrees).	Expected Resistance (k Ω).
1	0	$2.2 + (0 \div 180) \times 7.8 = 2.2$
2	30	$2.2 + (30 \div 180) \times 7.8 = 3.5$
3	45	$2.2 + (45 \div 180) \times 7.8 = 4.15$
4	90	$2.2 + (90 \div 180) \times 7.8 = 6.5$
5	105	$2.2 + (105 \div 180) \times 7.8 = 7.25$
6	130	$2.2 + (130 \div 180) \times 7.8 = 7.83$
7	145	$2.2 + (145 \div 180) \times 7.8 = 8.49$
8	180	$2.2 + (180 \div 180) \times 7.8 = 10.0$



Graph Selection & Explanation:

- Correct Graph: Graph A – Increases

Why the others are incorrect:

- **Graph B:** Flat Line – This graph shows no change in resistance, which is incorrect. The data clearly shows resistance increases with bend angle.
- **Graph C:** Decreases – This graph shows resistance dropping as bend increases, which is the opposite of what the sensor data shows.

Step by step instructions:

1. Introduce the Formula and Task Clearly.

- Present the resistance formula and explain each component: baseline resistance, bend angle, and scaling factor.
- Demonstrate how to calculate resistance for one or two bend angles as examples.

2. Step-by-Step Calculation Support.

- Guide students through completing the table row by row, allowing them to practice applying the formula.
- Provide calculators or digital tools to assist with calculations if needed.

3. Graphing the Data.

- Show students how to use the Canva Graph Maker or another graphing tool.
- Explain how to label axes correctly (X-axis: Bend Angle, Y-axis: Resistance).
- Encourage accurate plotting and neat presentation.

4. Data Analysis and Critical Thinking.

- Present the three sample graphs and explain their characteristics.
- Ask students to compare their plotted data with the sample graphs.
- Facilitate discussion or written responses explaining which graph matches their data and why the others are incorrect.

Possible strategies for implementation:

1. Peer Collaboration and Review.

- Encourage students to work in pairs or small groups to compare results and reasoning.
- Promote respectful discussion of different ideas and strategies.

2. Use Guided Questioning

- Ask: “How does resistance change as bend angle increases?”
 - “Why do you think a flat line or decreasing graph wouldn’t match the data?”
 - “What does the increasing graph tell us about sensor behavior?”

3. Differentiation and Scaffolding

- Provide formula breakdowns and examples for students needing extra support.
- Allow students to explain their reasoning verbally or through drawings if writing is challenging.



Builder

Objective:

Students will design and build a basic circuit in Tinkercad that connects multiple flex sensors to an Arduino. They will understand how hand movements are detected as signals

and translated into data outputs, linking physical action to digital reactions.

Answer:

Q1: When you write the code, what kinds of instructions would you give it?

- I would write instructions to read the analog values from each flex sensor.
- Then, I'd use if-else statements to detect specific bend positions.
- I would tell the Arduino to display a word or letter on the screen depending on the finger position.

Q2: What kinds of messages could be shown if this circuit was connected to a screen?

- The screen could show letters, like "A," "B," or "C," depending on the hand gesture.
- It could display words like "HELP," "YES," or "NO" to help someone communicate.
- It might even show emojis or actions, like a waving hand or a smiley face!

Circuit link:

<https://www.tinkercad.com/things/4zt3whPeNW0/editel?sharecode=2slwDETbOiILXsCaLFs31p6CXRm6gDrg1H67Y3xDEX4>



Components Needed in Tinkercad:

- Arduino Uno R3
- Breadboard (Small or Full)
- 3–5 Flex Sensors (use potentiometers as virtual equivalents, labeled as "Flex Sensor")
- LCD display (use the I2C 16x2 LCD if available, or standard LCD)
- Resistors (10kΩ, one per flex sensor)

Activity Instructions:

1. Step 1: Start the Tinkercad Circuit.

- Open Tinkercad Circuits
- Create a new project and drag in:
 - Arduino Uno
 - Breadboard
 - 5 flex sensors
 - 5 resistors
 - I2C LCD
 - Jumper wires

2. Step 2: Place Flex Sensors.

For each flex sensor:

- Connect one pin of the sensor to 5V (power rail).
- Connect the other pin to:
 - A 10kΩ resistor going to GND (ground rail).
 - An analog input pin on the Arduino:

1. Sensor 1 → A0
2. Sensor 2 → A1
3. Sensor 3 → A2
4. Sensor 4 → A3
5. Sensor 5 → A4

3. Step 3: Connect the LCD (I2C Version).

- Connect:
 - GND → GND (Arduino)
 - VCC → 5V (Arduino)
 - SDA → A4 (Arduino)
 - SCL → A5 (Arduino)

Note: You can explain that I2C helps reduce the number of pins used to control the screen.

4. Step 4: Wire Power and Ground Rails.

- Connect Arduino 5V to the positive rail on the breadboard.
- Connect Arduino GND to the ground rail on the breadboard.

5. Step 5: Review the Circuit.

- Have students check:
 - Each flex sensor has 1 wire to 5V, 1 to GND via resistor, and 1 to analog input.
 - LCD wiring matches I2C layout.
 - All power/ground rails are connected properly.

Tip: Explain that flex sensors change resistance when bent—just like potentiometers change resistance when turned.

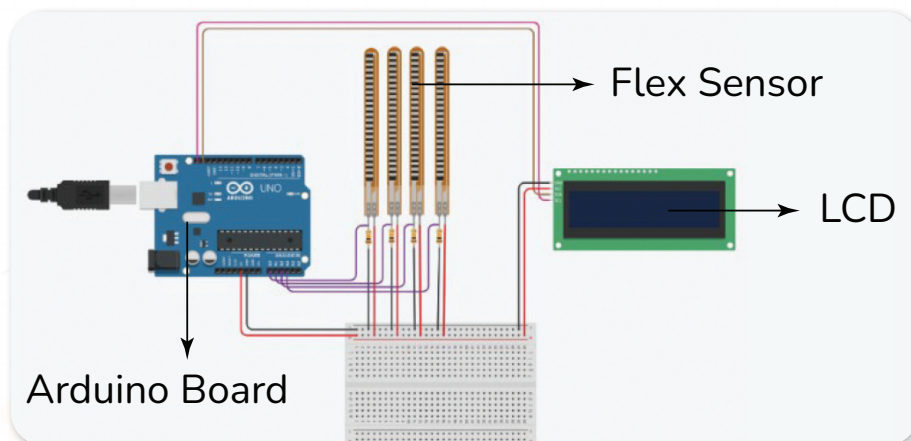


Figure 9: Arduino Setup with Ultrasonic Sensors and LCD Display.

Possible Strategies for Implementation:

1. Introduce the Task Clearly

- Explain the goal: to build a virtual circuit that senses finger bends and sends signals to an Arduino.
- Show a completed example or schematic diagram of the circuit for reference.

2. Guide Through Circuit Design.

- Walk students step-by-step through placing flex sensors, connecting wires, and linking to the Arduino in Tinkercad.
- Emphasize the importance of correct connections to analog inputs and ground.
- Explain how the flex sensor's bending changes resistance, which the Arduino reads.

3. Coding Planning Discussion

- Before coding, ask students to think about what instructions the Arduino needs (e.g., reading sensor values, mapping values to messages).
- Encourage brainstorming about message types that could be displayed on an LCD or serial monitor.

4. Hands-On Virtual Building and Testing.

- Allow students time to build, simulate, and troubleshoot their circuits.
- Encourage experimentation with different sensor inputs and outputs.

5. Collaborative Problem-Solving

- Facilitate peer discussions and troubleshooting sessions.
- Promote sharing of solutions and strategies.

6. Link to Real-World Applications.

- Discuss how such circuits are used in wearable tech, prosthetics, and assistive devices.

7. Support Diverse Learners.

- Provide circuit diagrams, wiring guides, or video tutorials.
- Allow code templates or partial code for students needing extra help.

Evaluate



Teacher's Role:

1. Coach for Independent Debugging.

- Encourage students to troubleshoot their code independently.
- Ask guiding questions like, "What is your code telling the LCD to do right now?" or "How do you know the message logic is working correctly?"

2. Clarifier of Concepts

- Reinforce understanding of if/else logic, sensor input readings, and string comparison.
- Help clarify variable usage and LCD function calls (e.g., lcd.print, lcd.clear).

3. Facilitator of Reflection

- Encourage students to reflect on how their code translates finger movement into communication.
- Support them in identifying possible improvements or additional signs to add.

Expected from Students:

1. Apply Logic to Code Smart Behavior.

- Use the if/else statements to detect combinations of bent fingers.
- Assign specific messages (like "Hello", "Stop", or "Help") based on flex sensor readings.

2. Test and Refine Their Code.

- Upload the code to the Tinkercad circuit.
- Check that the LCD correctly shows the message when fingers bend.
- Use the lastMessage variable to prevent repeated messages unless there's a change.

3. Demonstrate Understanding Through Code.

- Fill in the blank sections using variables, conditions, and print functions.
- Show that their glove “talks” by displaying the correct message for at least two different gestures.

4. Optional: Extension

- accuracy.

Parts of the book included.



Now it's time to code your circuit!

Your smart glove will display messages when fingers bend. Use what you've learned to complete the missing parts of the code.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x20, 16, 2); // Adjust if needed // Set up the LCD screen using the I2C address

const int FlexSensor[5] = {A0, A1, A2, A3, A4}; // Define analog pins for flex sensors
int values[5]; // Store sensor readings
const int threshold = 200; // Define threshold for detecting bend
String lastMessage = "";

void setup() {
  for (int i = 0; i < 5; i++) { // Set all flex sensor pins as input
    pinMode(FlexSensor[i], INPUT);
  }
  Serial.begin(9600); // Start serial monitor
  lcd.init(); // Start LCD and show welcome
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Smart Glove");
  delay(1500);
  lcd.clear();
}

void loop() {
  for (int i = 0; i < 5; i++) { // Read values from sensors
```

26



```
values[i] = analogRead(FlexSensor[i]);
}
String message = ""; // Create message variable
Your turn: fill in the blank.
```

```
// Use logic here to set the message
// based on bent fingers
// Example: if index + middle bent ->
// message = "More";
// Use values[0] for index, values[1] for
// middle, etc.
```

```
if ( _____ ) {
  message = " _____ ";
}
else if ( _____ ) {
  message = " _____ ";
}
```

```
// Add more conditions as needed
```

```
// -----
```

```
Your turn: fill in the blank.
```

```
// Display message if it's new
```

```
if ( _____ ) {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Sign Detected.");
  lcd.setCursor(0, 1);
  lcd.print( _____ );
  Serial.println( _____ );
  lastMessage = message;
}
// -----
```

```
delay(300); // Pause between reads
```

```
}
Now upload the code to the previous circuit and try it out!
```

STEAM SCOUTS

27



Showcase

Objective:

Students will apply their knowledge of flex sensors and Arduino programming to complete and upload code that allows a smart glove to display messages when fingers bend, demonstrating understanding of sensor input processing and output display.

Code:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Initialize the LCD with the I2C address (0x20 is common, adjust if yours is different, e.g., 0x27)
LiquidCrystal_I2C lcd(0x20, 16, 2);

// Flex sensor analog pin definitions (assuming standard finger mapping)
// A0: Index Finger Flex Sensor
// A1: Middle Finger Flex Sensor
// A2: Ring Finger Flex Sensor
// A3: Pinky Finger Flex Sensor
// A4: Thumb Flex Sensor
const int FlexSensor[5] = {A0, A1, A2, A3, A4};
int values[5]; // Stores analog readings from flex sensors

// IMPORTANT: CALIBRATE THIS THRESHOLD FOR YOUR SPECIFIC SENSORS!
// A reading BELOW this threshold means the finger is considered 'bent' (flexed).
// If bending all fingers doesn't trigger "Stop", increase this value (e.g., to 250 or 300)
// If single finger bends trigger unintended messages, decrease this value.
const int threshold = 200;

// Variables to prevent repeated messages from continuously displaying for the same gesture
String lastMessage = "";

void setup() {
  // Set all flex sensor pins as INPUT
  for (int i = 0; i < 5; i++) {
    pinMode(FlexSensor[i], INPUT);
```

```

}

// Initialize Serial communication for debugging (useful to see raw sensor values or
messages)
Serial.begin(9600);

// Initialize and light up the LCD
lcd.init();
lcd.backlight();
lcd.setCursor(0, 0);
lcd.print("Smart Glove");
delay(1500); // Display initial message briefly
lcd.clear(); // Clear the screen after the welcome message
}

void loop() {
  // Read analog values from all flex sensors
  for (int i = 0; i < 5; i++) {
    values[i] = analogRead(FlexSensor[i]);
    // Uncomment the line below for live debugging of sensor values on Serial Monitor
    // Serial.print("F"); Serial.print(i); Serial.print(":"); Serial.print(values[i]); Serial.print(" ");
  }
  // Serial.println(); // Uncomment for debugging, adds a newline after all sensor values

  String message = ""; // Variable to store the detected gesture message for this loop
  iteration

  // --- Gesture Detection Logic (Ordered from most complex to simplest) ---
  // It's crucial to check more complex gestures (involving more fingers) FIRST.
  // This prevents simpler, overlapping gestures from being detected instead.

  // 1. All fingers bent (Index + Middle + Ring + Pinky + Thumb) -> "Stop"
  if (values[0] < threshold && values[1] < threshold && values[2] < threshold && values[3]
  < threshold && values[4] < threshold) {
    message = "Stop";
  }
}

```

```
// 2. Index + Middle + Thumb -> "Love"
// NOTE: The request lists "Index + Middle + Thumb" for both "Love" and "Go".
// Only one can be detected if the conditions are identical.
// This implementation prioritizes "Love". To distinguish "Go", you would need a different
gesture (e.g., speed, or another sensor).
else if (values[0] < threshold && values[1] < threshold && values[4] < threshold) {
  message = "Love";
}
// 3. Index + Middle -> "More"
else if (values[0] < threshold && values[1] < threshold) {
  message = "More";
}
// 4. Middle + Ring -> "Help" (Changed from "Please" in the original code, as per request)
else if (values[1] < threshold && values[2] < threshold) {
  message = "Help";
}
// --- Single Finger Bends ---
// These are checked last and with 'else if' to ensure they only trigger if no complex
gesture is detected.
// Assuming this mapping for single bends:
else if (values[0] < threshold) { // Index Finger
  message = "Yes";
} else if (values[1] < threshold) { // Middle Finger
  message = "Fine"; // New message as per request
} else if (values[2] < threshold) { // Ring Finger
  message = "No";
} else if (values[3] < threshold) { // Pinky Finger (Assumed this maps to "Thank You" for
single bends)
  message = "Thank You"; // New message/mapping as per request
}
// If "Go" is meant to be a single finger bend (e.g., Thumb alone), you would add:
// else if (values[4] < threshold) { // Thumb
//   message = "Go";
// }
// As per your request, "Go" was for Index+Middle+Thumb, which conflicts with "Love".
```



```

// --- "All Done" Gesture (Transition state) ---
// This logic detects a sequence: all fingers were bent, and then all fingers are released.
// This is separate from static gesture detection, as it relies on a state change.
static bool allBentBefore = false; // Tracks if all fingers were bent in the *previous* loop
iteration
    bool allBentNow = true;        // Assumes all are bent unless proven otherwise in current
iteration
    for (int i = 0; i < 5; i++) {
        if (values[i] > threshold) { // If any finger is NOT bent
            allBentNow = false;
            break; // No need to check further, not all fingers are bent
        }
    }
    // If all fingers were bent in the *previous* loop AND now they are *all released*
    if (allBentBefore && !allBentNow) {
        message = "All Done";
    }
    allBentBefore = allBentNow; // Update state for the next loop iteration

// --- Display and Update Logic ---
// Only update the LCD and Serial Monitor if a NEW message is detected
// This prevents rapid flickering and redundant output.
if (message != "" && message != lastMessage) {
    lcd.clear();        // Clear the LCD screen
    lcd.setCursor(0, 0); // Set cursor to the first line (row 0)
    lcd.print("Sign Detected:");
    lcd.setCursor(0, 1); // Set cursor to the second line (row 1)
    lcd.print(message); // Display the detected message
    Serial.println(message); // Also print to Serial Monitor for debugging
    lastMessage = message; // Store the current message as the last one displayed
}
// If no new gesture is detected or the message is the same as before, the LCD keeps
showing the last message.
// You might consider adding a timeout to clear the screen if no gesture is held for a period.

delay(300); // Small delay to debounce sensor readings and prevent rapid message re-

```

triggering

}

Activity Instructions:

1. Step 1: Review the Goal.

- Explain to students:
 - “Each sensor will detect if a finger is bent.”
 - “You’ll write code to decide which fingers are bent, and what message that means.”
 - “If a certain combination is detected, the glove will display a word on the screen.”

2. Step 2: Scaffolded Coding.

- Walk students through each block:
 - Setup: sensors as input, LCD init
 - Loop: read sensor values
 - Use if conditions to map sensor values to messages

Encourage them to start with one gesture and build from there (e.g., just the index finger bent = “Yes”).

3. Step 3: Threshold Thinking.

- Explain the role of threshold:
 - Values above it mean the finger is bent.
 - Use logic like if (values[0] > threshold) to detect that.

4. Step 4: Upload and Simulate.

- Students simulate bending sensors (adjust values in Tinkercad).
- Test and see if correct message appears on screen.

5. Step 5: Challenge Extensions.

- Add more conditions (2 or 3 fingers at once = longer words).
- Customize the LCD display (add name, emoji words, etc.).
- Use lastMessage logic to avoid repeating messages unnecessarily.

Note to Teachers:

You can have students create their own gesture-to-message mappings to promote creativity, accessibility awareness, and real-world thinking.

Possible Strategies for Implementation:

1. Review Prior Learning

- Recap key coding concepts and circuit connections used in the Builder activity.
- Highlight how sensor readings translate into messages on the LCD.

2. Code Walkthrough

- Guide students through the provided code template, explaining each section,

especially the parts to be completed.

- Clarify logical structures like if-else conditions for detecting bent fingers.

3. Scaffold Coding Support

- Encourage students to fill in missing code based on sensor values and bending thresholds.
- Provide hints or examples for coding conditions and message displays.
- Promote debugging strategies such as checking serial monitor outputs.

4. Peer Collaboration and Sharing

- Facilitate pair programming or small group code reviews.
- Encourage students to test and improve each other's code.

5. Testing and Iteration

- Have students upload code to their circuit and observe the LCD output.
- Encourage iterative testing, refining conditions, and expanding message options.

6. Reflection and Presentation

- Allow students to demonstrate their smart glove functionality to the class.
- Discuss challenges faced and problem-solving approaches.

Rubric for Assessment (Total 100 points):

Criteria	Excellent (90-100 Points)	Good (70-89 Points)	Needs Improvement (50-69 Points)	Poor (<50 Points)	Points
Technical Accuracy	Code runs without errors, all sensors respond correctly, messages display appropriately.	Minor bugs; most sensors work; messages mostly correct.	Several errors affecting functionality; partial messages.	Code does not run or sensors unresponsive.	/40
Creativity	Messages are clear, varied, and creatively designed.	Messages mostly clear with some variety.	Messages repetitive or unclear.	Messages missing or not relevant	/20
Code Logic & Structure	Logical and efficient use of conditions and loops; well-commented.	Mostly logical with minor inefficiencies; some comments.	Some logical errors or poor structure; few comments.	Code lacks logic or organization.	/20
Presentation & Reflection	Student confidently explains code and problem-solving; answers questions well.	Adequate explanation and reflection.	Limited explanation; struggles with questions.	No explanation or reflection.	/20



Now I Can

Objective:

Students will reflect on their learning and identify how they've grown in understanding communication, empathy, and the role of assistive tools in helping others share ideas.

Activity Instructions:

1. Introduce the Reflection

- Say: "Let's look back on everything we've explored this lesson. You learned how it feels to face communication challenges, and how tools can help people share their thoughts in special ways."

2. Guide Students Through the Checklist.

- Read each statement aloud one at a time.
- After each one, ask students to give a thumbs up, sideways, or down to show how confident they feel.
- Example prompts:
 - "Did you learn how people share ideas without talking?"
 - "Do you know how tools can help people talk?"

3. Student Completion

- Let students check off or color the "Now I Can" boxes they feel good about.
- Remind them: "It's okay if you didn't check all of them. We're all still learning."

4. Optional Extension

- Ask students to circle the one they're most proud of.
- Invite them to write a sentence or draw a picture to show their favorite learning moment.

Relation to Standards

1. NGSS (Next Generation Science Standards).

- **MS-LS1-8:** Gather and synthesize information about how the nervous system controls movement.
- **MS-PS4-3:** Integrate qualitative scientific and technical information to support the explanation of how sensors detect and convert signals.
- **MS-ETS1-2:** Evaluate competing design solutions based on criteria and constraints.

2. CCSS (Common Core State Standards).

- **CCSS.MATH.CONTENT.6.SP.B.5:** Summarize numerical data sets in relation to their context.

- **CCSS.ELA-LITERACY.SL.6.4:** Present claims and findings clearly, sequencing ideas logically.

3. ISTE (International Society for Technology in Education).

- **Innovative Designer:** Students use technology and design processes to solve problems.
- **Computational Thinker:** Students develop and employ strategies for understanding and solving problems in ways that leverage technology.